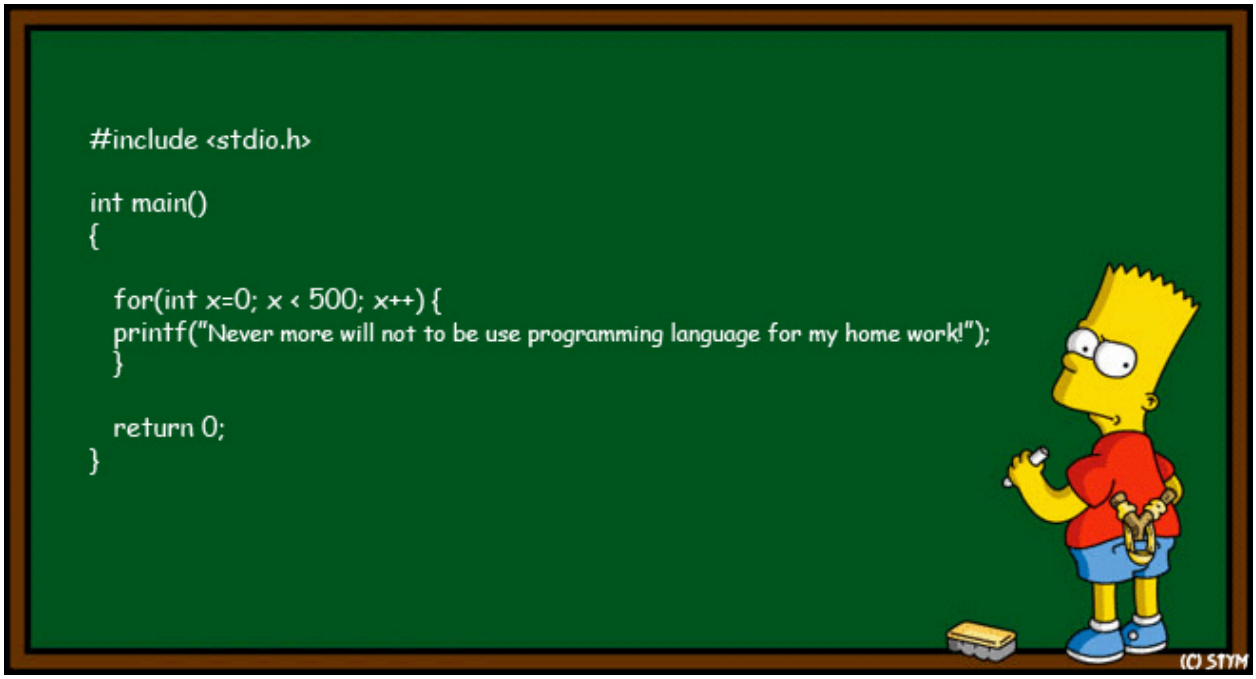
UT2 Practica 1



Actividad 1. **4**
Actividad 2: **4**
Actividad 3. **5**
Principales Características 5
Plugin Seleccionado: Pylance 6
Actividad 4. Depuración de código (modo Debug) y puntos de ruptura (breakpoints) **8**

Actividad 1.

Empezamos con la práctica guiada con Visual Studio Code: Instalación del IDE (windows, linux).
Visitamos: https://code.visualstudio.com/
Nos va a detectar el S.O. que tenemos y solamente tendremos que descargarnos el fichero de instalación que nos interese.

Respuesta: Ya tenemos el Visual Studio Code instalado desde el primer día así que supongo que no tengo que instalarlo otra vez.

Actividad 2:

Para la realización de estás actividades prácticas hay disponibles una serie de scripts (los ficheros adjuntos a la actividad en el comprimido “DepurarParte1.zip”) con los que trabajaremos. Trata de ejecutar el primer script (script_1) y contesta a las preguntas.
¿En qué lenguaje de programación está escrito?

- Esta en python

¿Es un lenguaje compilado o interpretado?

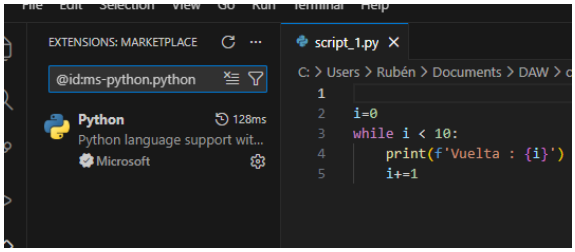
- Es un lenguaje interpretado

¿Has podido ejecutarlo directamente nada más instalar el Visual Studio Code?

- Como ya tenía el VSC instalado no tengo claro si me ha hecho falta instalar algo para abrir el archivo, pero para ejecutar el código si, hizo falta una extensión.

¿Qué tienes que instalar para ejecutarlo?

- Una extensión llamada python que te abre una terminal dentro del vsc con el codigo ejecutado.

- 

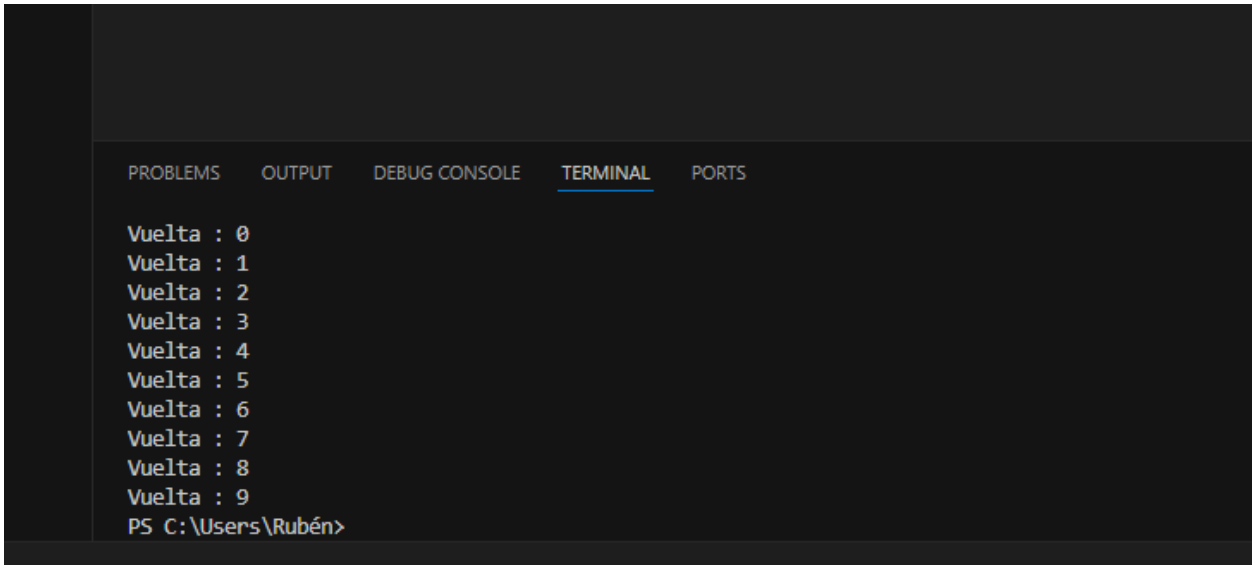
¿Has tenido que instalar algún plugin adicional?

- No, solo el llamado python

¿Cual?

- Ninguno

(adjunta capturas de pantalla para documentar lo que has hecho en cada paso hasta conseguir ejecutar el código)



Actividad 3.

Busca información sobre el lenguaje en el que están escritos los scripts adjuntos y pon sus principales características, sus puntos fuertes, sus puntos débiles y principales campos de aplicación a día de hoy en el mundo de las tecnologías de la información. Busca plugins que te puedan ayudar a trabajar de forma más eficiente e instala alguno (al menos uno)

Python es un lenguaje de programación de alto nivel, interpretado y multiparadigma, cuya filosofía se centra en la legibilidad del código. En 2026, seguirá siendo el lenguaje más popular del mundo gracias a su versatilidad y el innecesario uso de la Inteligencia Artificial.

Principales Características

- **Sintaxis Limpia:** Utiliza la indentación (sangría) para delimitar bloques de código en lugar de llaves ({}), lo que obliga a escribir código ordenado.
- **Tipado Dinámico:** No necesitas declarar el tipo de una variable (entero, texto, etc.) al crearla; el intérprete lo deduce en tiempo de ejecución.
- **Interpretado:** El código se ejecuta línea a línea, lo que facilita enormemente la depuración y el aprendizaje.
- **Multiparadigma:** Soporta programación orientada a objetos, imperativa y, en menor medida, funcional.

Puntos Fuertes	Puntos Débiles
Velocidad de desarrollo: Se escribe mucho menos código que en Java o C++ para la misma tarea.	**Velocidad de ejecución:** Al ser interpretado, es más lento que lenguajes compilados (aunque esto se mitiga con librerías en C).
Ecosistema inmenso: Tiene librerías para casi cualquier cosa (PyPI tiene cientos de miles de paquetes).	**Consumo de memoria:** No es el lenguaje más eficiente en gestión de recursos de hardware.
Comunidad global: Es casi imposible encontrar un problema que alguien no haya resuelto ya en foros.	**Desarrollo Móvil:** Aunque existen frameworks como Kivy, no es la opción estándar para apps de iOS o Android.

¿Cuál/es has instalado?

Plugin Seleccionado: Pylance

- **Pylace** no tiene que instalarse por separado, viene incluido en el plugin “python” que instalé anteriormente en esta misma práctica guiada.

¿Para qué sirven?

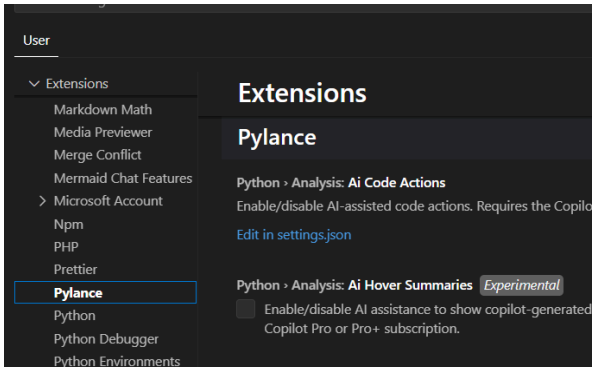
Pylace Es el "cerebro" para Python en VS Code. Proporciona IntelliSense (autocompletado inteligente), comprobación de errores en tiempo real, resaltado de sintaxis avanzado y sugerencias de tipos.

Documenta todo el proceso y el plugin funcionando (es decir cómo estaba el entorno antes y después de aplicar el plugin, que se vea la funcionalidad del plugin).

Al ser el mismo plugin que anteriormente ya envié las capturas de pantalla con los cambios pero puedo describirlos más detalladamente.

Primero, añade un botón ▶ que funciona similar al live server de javascript. Ejecutando el código y mostrando el resultado. Pero como Python no ejecuta en un navegador este botón abre una terminal dentro del propio vsc para enseñarte el código funcionando.

También he podido confirmar que lo tengo en las opciones del VSC

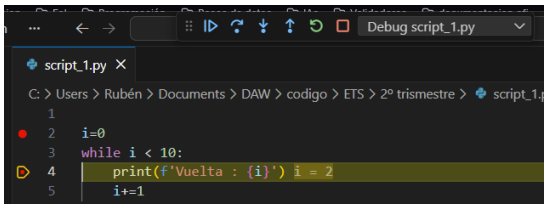


Para terminar comento las mejoras que el plugin ofrece y no son visibles realmente:

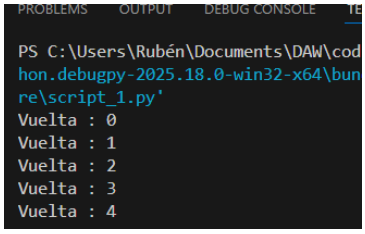
1. **IntelliSense Veloz:** Autocompletado de código mucho más rápido que los motores antiguos (como Jedi).
2. **Información de Tipos:** Al pasar el ratón sobre una variable o función, te muestra qué tipo de dato es (int, str, list, etc.) gracias a su motor basado en *Pyright*.
3. **Auto-Importaciones:** Si empiezas a usar una librería que no has importado (ej. `math`), Pylance te ofrece añadir automáticamente el `import math` al principio del archivo.
4. **Detección de Errores en Vivo:** Subraya en rojo errores de sintaxis o variables no definidas antes de que intentes ejecutar el código.

Actividad 4. Depuración de código (modo Debug) y puntos de ruptura (breakpoints)

4. 1. Pon un punto de ruptura en el script_1.py donde creas conveniente y ejecuta línea a línea viendo como se va ejecutando el código y los valores que van imprimiendo por pantalla. Prueba con las 4 opciones de ejecución (continue, step over, step into, step out).

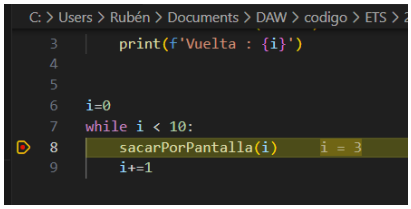


¿Qué diferencias hay entre los 4 modos de ejecución para este caso?
Las diferencias vienen a ser las mismas, se puede ir línea a línea, se puede ejecutar de golpe hasta el punto de ruptura y se puede cancelar.



4.2. Ahora ejecuta el script_2.py y coloca nuevamente al menos un punto de ruptura. Prueba con las 4 opciones de ejecución (continue, step over, step into, step out).

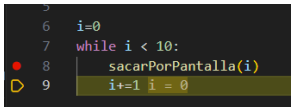
¿Qué diferencias hay entre los 4 modos de ejecución?
(Saca capturas de pantalla dónde se vean las diferencias si es que las hay)



Usando el modo **continúe** al tener el punto de ruptura en la llamada a la función da las vueltas de una en una en cada click

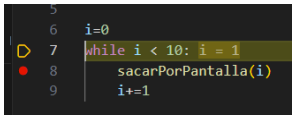
Usando el modo Step Over da las vueltas a base de click por línea, unos 3 clicks cada vuelta.

Click 1



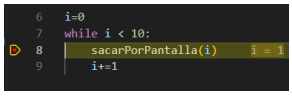
Vuelta 0

Click 2



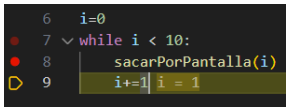
Vuelta 0

Click 3



Vuelta 0

Click 4

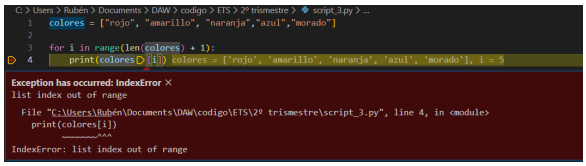


Vuelta 1

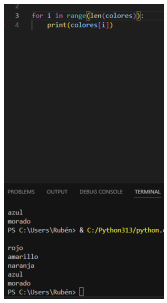
4.3. Ejecuta el script_3.py en modo normal. Saca captura de pantalla de lo que ocurre. ¿Qué está ocurriendo?

- Esta saltando un error en la línea 4 “print(**colores[i]**)”

Pon un punto de ruptura y ejecuta paso a paso para dar con el error y solucionarlo
¿Qué has hecho para solucionarlo?



Básicamente el error es que en el bucle le estamos sumando un +1 al array por lo que al no haber más array que imprimir da error.



Sin el +1 en el bucle no salta error, no he tenido que hacer ningún paso para encontrarlo, era evidente. Pero asumo que lo normal habría sido ir dando clicks al step over hasta ver que en la última vuelta no le queda colores que imprimir.

4.4. Ejecuta el script_4.py
¿Se produce algún error?

- Si *“Can only concatenate str (not “int”) to str”*

¿Por qué?

- Pues el primer error es que python no sabe que significa colores[i] +1. Por qué “colores[i] es un texto “rojo” por ejemplo. Y rojo+1 no significa nada. Yo eliminaría ese +1 por qué no entiendo qué quiere conseguirse con eso. Y después al eliminarlo pasa lo mismo que en el ejercicio anterior. El color+1 busca un color adicional que no existe. Lo elimino tambien.

Explica que está ocurriendo.

```
C: > Users > Rubén > Documents > DAW > codigo > ETS > 2º trismestre > script_4.
1  colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3  for i in range(len(colores)):
4      resultado=colores[i]
5      print(resultado)
```

- Me imagino que lo que se quería conseguir era un resultado más limpio, guardando colores[i] dentro de una variable y después imprimir solo la variable en cada vuelta. No entiendo los +1 en este código. Pero así como en la captura de pantalla imprime el array sin problemas.

```
rojo
amarillo
naranja
azul
morado
PS C:\Users\Rubén>
```

4.5. Ejecuta el script_5.py paso a paso fíjate en los valores que van tomando las variables (situándose encima con el ratón y también observando en la parte izquierda en la sección de “Variables”).
Comenta lo que va haciendo el código y documenta lo que ves

- El código define la función **operaciones(a,b)** la cual indica que tendrá dos valores, a y b
- Salta a resultado:operaciones = **operaciones(5,10)** //Aquí es donde se le da valor a (a y b)
- Para calcular la función **operaciones(5,10)** vuelve a la línea 3 y empieza a calcular
- Resultado_intermedio = a*2 **(5*2=10)**
- Resultado_intermedio2 = resultado_intermedio*10 **(10*10=100)**
- Resultado_final = resultado_intermedio + b + resultado_intermedio2 **(10 + 10 + 100= 120)**
- Vuelve a la línea 8, le da a resultado_operacion el calor de la función **(120)**
- Baja a la línea 9 e imprime el resultado.

(que tipo de ejecuciones has hecho: step over, step into,etc).

- He tirado de Step into todo el rato, porque Step over saltaba al resultado sin explicarme cómo conseguía el valor.

4.6. Ejecuta el script_6.py. Este script como verás, se parece al anterior con la diferencia de que es el usuario el que pasa por teclado el valor de los operandos. Prueba a ejecutar el script con varios valores.
¿Qué pasa si no le metes un número y le metes letras?

- Pasa lo mismo en ambas casuísticas, el código está escrito para pedir al usuario un “int” que es un número entero en base 10. Si recibe algo que no sea eso no ejecuta el código.
- A mi parecer le falta un código de error claro para un usuario.
  “if (p1 == !int && p2 == !int) print(“esto no es un número válido”)”

(Documenta el proceso como has hecho anteriormente donde quede reflejado todas las pruebas que has hecho).

Intento1

```
PS C:\Users\Rubén> & C:/Python313/py
Introduce el primer operando: 2
Introduce el segundo operando: 13
Resultado final: 57
PS C:\Users\Rubén>
```

Dos números simples dan el resultado sin problemas

Intento2

```
Introduce el primer operando: 9999999999
Introduce el segundo operando: 999999999999
Resultado final: 1219999999977
PS C:\Users\Rubén>
```

Dos números desorbitados dan el resultado sin problemas

Intento3

```
Introduce el primer operando: 4
Introduce el segundo operando:
Traceback (most recent call last):
  File "c:\Users\Rubén\Documents\DAW\codigo\ETS\2º trismestre\script_6.py", line 10, in <module>
    p2 = int(input("Introduce el segundo operando: "))
ValueError: invalid literal for int() with base 10: ''
PS C:\Users\Rubén>
```

Poner solo un numero salta el error “invalid literal for int() with base 10: “”
Lo que entiendo es que los valores que solicita (int) son solo números y en decimal.

Intento4

```
Introduce el primer operando: 34
Introduce el segundo operando: a
Traceback (most recent call last):
  File "c:\Users\Rubén\Documents\DAW\codigo\ETS\2º trismestre\script_6.py", line 10, in <module>
    p2 = int(input("Introduce el segundo operando: "))
ValueError: invalid literal for int() with base 10: 'a'
PS C:\Users\Rubén>
```

Viene a ser lo mismo que el intento 3, al no ser un número en base 10 no lo detecta.

4.7. Ejecuta el script_7.py en modo normal para entender qué hace el programa.
En teoría el usuario introduce por teclado un número y el programa calcula si es un número par o impar; después pregunta si quiere salir de la ejecución o si quiere seguir. Si se introduce una ‘s’ se sale de la ejecución; si se introduce una ‘n’ sigue pidiendo al usuario que introduzca otro número.

Prueba a introducir distintos valores por teclado para poner a prueba el programa (documenta indicando los valores que has probado con capturas de pantalla).

¿Funciona correctamente?
- No, sale de la ejecución se escriba lo que se escriba

¿Hay algún comportamiento que no debería tener?
- Si, debería de seguir pidiendo números hasta que el usuario introduce “y” cuando se lo pide.

¿Si en lugar de poner ‘y’ o ‘n’ ponemos lo mismo pero con mayúsculas qué pasa?
- El código se ejecuta sin problemas aunque se le conteste en mayúsculas

```
Quieres salir? y/n
N
Introduce un número:
```

¿Por qué?
- Deduzco que no es sensible a mayusculas

Prueba a poner puntos de ruptura y ejecutar el código línea a línea para dar con el error y solucionarlo.

- El error era que en esta parte “`while p2.lower() != "n":`“ la “n” deberia ser “y” y en esta “`p2 = "y"`” la “y” debería ser “n”.

Fin.